



# CIS 1902: Python

## Week 2

Ash Fujiyama

Slides adapted from Jorge Mendez, Arun  
Kirubarajan, and Tony Liu

# Scheduled Topics

- Logistics
- References and arguments (cont'd)
- Namespaces
- Exceptions
- Comprehensions



# Logistics

- HW0 and WS1 due next Monday (9/28) @ 11:59 pm
- WS1 released: list comprehensions
- Weekly OH:
  - Ash: Friday 3-5 pm\*
  - Foram: 7-8:30 PM Thursdays (Zoom)

# Style Grading

- All homeworks will be style graded!
- We will follow PEP8
- There are Python libraries to style-check, but Foram and I will be grading by hand
  - Quality of comments + documentation
  - More feedback on subjective metrics like “readability”
- The next few slides will cover some of the important stuff (for HW 0) but I’ll publish a proper style-guide sometime this week

# Style Grading: Names

- Naming conventions:
  - Variables are in **lowercase\_with\_underscores**
  - Functions are in **lowercase\_with\_underscores**
  - Classes are in **UpperCamelCase**
  - File names (module names) are in **lowercase\_with\_underscores**
  - Private instance/class variables and methods start with an underscore (eg. **`_var`**)
- Avoid non intuitive variable names (unless you provide a comment that provides the needed context)
  - **`for c in word`**: OK! (it's easy to see 'c' is short for 'character')
  - **`m = 3`**: vague, potentially confusing if used many lines later
  - **`foo = 3`**: not single character, but still not descriptive

# Style Grading: Keep in mind

- Include a docstring for each helper method you create
  - Follow what's already done in the hw
- Comments should be used to explain more complex code, and should not be stating the obvious
  - Use block comments by default
  - Inline comments should only be used when the comment is specific to one line
- No commented out code + no TODO comments

# Style Grading: Misc.

- Evaluating conditionals
  - `if first_condition == True:` Bad!
  - `if first_condition is True:` Bad!
  - `if first_condition:` More concise, good!
- Spacing: make 1 indent = 4 spaces (recommended)
  - I'm not checking this, but this is the general practice.
  - Bare minimum: pick either tabs or spaces and stick with it
- Keep lines < 120 characters long
- Don't mash code blocks together: `if first_condition: print("hi")`

# Style Grading: Misc.

- Don't mash code blocks together
  - `if first_condition: print("hi") # bad!`
  - `if first_condition:`  
 `print("hi")` `# good!`
- Don't switch between " (double quote) and ' (single quote), pick one!
  - But use double quotes for any triple-quoted strings, eg docstrings

# Equality: is vs ==

**is**: checks whether two references point to the same object (referential equality)

- Two objects with the exact same content are not referentially equal

**==**: compares two objects as defined by the object's type (structural equality)

- Each type has its own way of defining "equal"
- Usually based on comparing some values between two objects

# Equality: is vs ==

`is`: checks whether two references point to the same object (referential equality)

`==`: compares two objects as defined by the object's type (structural equality)

| Code                                                                           | Referential equality check                       | Structural equality check                        |
|--------------------------------------------------------------------------------|--------------------------------------------------|--------------------------------------------------|
| <code>a = [1,2]</code><br><code>b = [1,2]</code>                               | <code>a is b</code> returns <input type="text"/> | <code>a == b</code> returns <input type="text"/> |
| <code>a = [1,2]</code><br><code>b = [2,1]</code>                               |                                                  |                                                  |
| <code>a = [1,2]</code><br><code>b = a</code>                                   |                                                  |                                                  |
| <code>a = {'J' : 11, 'Q' : 12}</code><br><code>b = {'Q' : 12, 'J' : 11}</code> |                                                  |                                                  |

# Equality: is vs ==

`is`: checks whether two references point to the same object (referential equality)

`==`: compares two objects as defined by the object's type (structural equality)

| Code                                                                       | Referential equality check        | Structural equality check         |
|----------------------------------------------------------------------------|-----------------------------------|-----------------------------------|
| <code>a = [1,2]</code><br><code>b = [1,2]</code>                           | <code>a is b</code> returns False | <code>a == b</code> returns True  |
| <code>a = [1,2]</code><br><code>b = [2,1]</code>                           | <code>a is b</code> returns False | <code>a == b</code> returns False |
| <code>a = [1,2]</code><br><code>b = a</code>                               | <code>a is b</code> returns True  | <code>a == b</code> returns True  |
| <code>a = {'J': 11, 'Q': 12}</code><br><code>b = {'Q': 12, 'J': 11}</code> | <code>a is b</code> returns False | <code>a == b</code> returns True  |

# Equality: Changing Mutables

```
a = [1,2,3,4]
b = a
b[3] = 3
print(b)
print(a is b)
print(a == b)
```

Console Output:

[1, 2, 3, 3]

True

True

# Equality with Immutables: None

```
a = None  
b = None  
print(a == b)  
print(a is b)
```

Console Output:

True

True

# Equality with Small Ints

```
a = 42  
b = 42  
print(a == b)  
print(a is b)
```

Console Output:

True

True

# Equality with Immutables: Strings

```
a = "yam"  
b = "yam"  
print(a is b)  
print(a == b)
```

Console Output:

False

True

# Equality with Immutables: Strings

```
a = "taro"  
b = a  
print(a)  
print(a is b)  
print(a == b)
```

Console Output:

taro

True

True

# Equality with Immutables: Strings

```
a = "taro"  
b = a  
a = a + " root"  
print(a)  
print(a is b)  
print(a == b)
```

Console Output:

taro root

False

False

# Equality with Immutables: Strings

```
a = "taro root"  
b = a  
a = "t" + a[1:]  
print(a)  
print(a is b)  
print(a == b)
```

Console Output:

taro root

False

True

# Dynamic (Duck) Typing

```
def add_one_to_first_element(obj):  
    obj[0] += 1
```

```
foo = {1 : 1}  
bar = [1, 1]
```

```
add_one_to_first_element(bar)  
print(bar) # [2, 1]
```

```
add_one_to_first_element(foo) # does this work?  
print(foo)
```

```
foo[0] = 1  
add_one_to_first_element(foo) # does this work?  
print(foo)
```

Duck typing: “If it looks like a duck and quacks like a duck, then it’s a duck.”

Translation: Python won’t care about type of `obj` until `obj` is operated on. If the operation is not supported by `obj`’s type, then Python complains!

Question: which line raises the exception?

The slide features a dark teal background with several vertical dashed lines in a lighter teal color. There are four lines on the left side, one on the right side, and two on the far right side. The lines are composed of small squares.

# More on Functions

# Positional and Keyword Arguments

- You're probably already familiar with positional arguments: pass in your arguments in a specific order to the invoked function

```
def sub(a, b):  
    return a - b
```

```
sub(5, 3) → sub(a = 5, b = 3) → 5 - 3 → 2
```

```
sub(3, 5) → sub(a = 3, b = 5) → 3 - 5 → -2
```

# Positional and Keyword Arguments

- Keyword arguments explicitly bind passed values to the function arguments

```
def sub(a, b):  
    return a - b
```

`sub(5, 3)` → `sub(a=5, b=3)` → `5 - 3` → `2`

`sub(b=3, a=5)` → `5 - 3` → `2`

# Positional and Keyword Arguments

- Keyword arguments explicitly bind passed values to the function arguments
- Can make arguments optional by providing a **default value**

```
def sub(a, b=1):  
    return a - b
```

`sub(5, 3)` → `sub(a=5, b=3)` → `5 - 3` → `2`

`sub(5)` → `5 - 1` → `4`

`sub(5, b=3)` → `5 - 3` → `2`

# Positional and Keyword Arguments

- Keyword arguments explicitly bind passed values to the function arguments
- Can make arguments optional by providing a **default value**
- All required arguments must come before the first optional argument

```
def sub(a, b=1): # legal
    return a - b
```

```
def sub(a=1000, b): # illegal, will raise SyntaxError
    return a - b
```

# Example: is this valid?

```
def add(a, b):  
    return a + b  
  
point = (5, 6)  
  
print(add(point))
```

# Positional Unpacking

- `*` operator: can be used to explicitly pack or unpack a sequence in a function call or definition
- Argument assignment is by position, no keywords used!

```
def add(a, b):  
    return a + b
```

```
point = (5, 6)
```

```
# throws an error: treats point as first argument (a), thinks b is missing
```

```
print(add(point))
```

```
# now we "unpack" the point tuple into the positional arguments
```

```
print(add(*point)) # 11
```

# Positional Packing

- `*` operator: can be used to explicitly pack or unpack a sequence in a function call or definition
- Let's now look at an example of using `*` on a function definition

```
def print_everything(*args):  
    """prints all positional arguments"""  
    print(type(args))  
    print(args)  
    for arg in args:  
        print(arg)
```

```
print_everything(1, 2, "5", {}, 3)
```

## Console Output:

```
<class 'tuple'>  
(1, 2, '5', {}, 3)  
1  
2  
5  
{  
}
```

# Positional Packing

- `*` operator: can be used to explicitly pack or unpack a sequence in a function call or definition
- If we already have a sequence, we need to explicitly unpack it to pass it into this function

```
def print_everything(*args):  
    print(type(args))  
    print(args)  
    for arg in args:  
        print(arg)  
  
print_everything(*"banana")
```

## Console Output:

```
<class 'tuple'>  
("b", "a", "n", "a", "n", "a")  
b  
a  
n  
a  
n  
a
```

# Positional Packing

- `*` operator: can be used to explicitly pack or unpack a sequence in a function call or definition
- If we already have a sequence, we need to explicitly unpack it to pass it into this function

```
def print_everything(*args) :  
    print(type(args))  
    print(args)  
    for arg in args:  
        print(arg)  
  
print_everything(*"banana")  
print_everything("banana")
```

## Console Output:

```
<class 'tuple'>  
("b", "a", "n", "a", "n", "a")  
b  
a  
n  
a  
n  
a  
<class 'tuple'>  
("banana")  
banana
```

# Keyword Unpacking

- `**` operator explicitly packs/unpacks a **dictionary** in a function call or definition
- Argument keywords now must match!

```
def sub(a, b):  
    return a - b
```

```
print(sub(a=1, b=2)) # -1
```

```
dict1 = {'c': 1, 'b': 2}
```

```
print(sub(**dict1)) # TypeError: add() got an unexpected keyword argument 'c'
```

```
dict2 = {'b': 4, 'a': 5}
```

```
print(sub(**dict2)) # 1
```

```
print(sub(a=dict2['a'], b=dict2['b'])) # 1
```

# **\*args and \*\*kwargs**

- If you don't have a more suitable argument name, \*args is used in function signature to hold positional arguments, and \*\*kwargs for keyword arguments

# More Keyword Unpacking Examples

# When to Pack/Unpack

- **Function Signature/Definition:** Include packing operator when you expect arbitrary number of arguments to be passed in
- **Function Calling:** Include unpacking operator if you have a sequence or dictionary and you want to pass in its elements *individually*

Examples:

- `print()` takes in an arbitrary number of arguments
- `def process_team(team_name, *team_members)`
- `def create_new_employee(first_name, last_name, job_id, **details)`

# Documentation

You can maybe see how function definitions can get a little bit complicated

```
def fetch_report(report_id, *sections, format = "pdf", **options)
```

- report\_id is positional-only
- sections: arbitrarily sized tuple containing the other positional arguments
- format is the first mandated keyword argument
- options: arbitrarily sized dictionary containing optional flags

By default, your functions should already have docstrings, but when using `*` and `**`, ensure you are explicit about what keyword parameters are valid

# Namespaces

# Namespaces

- But we don't usually need to explicitly state the namespace - why is that?
- **Scope:** a region where namespace attributes are directly accessible without using the `namespace .` operator
- Order of resolution (searching namespace for a particular attribute):
  1. **Local scope:** code block of current function
  2. **Enclosing scope:** block of nested function (rare)
  3. **Global scope:** top-level scope in your Python program/file
  4. **Built-in scope:** special scope containing all built-ins

# Namespaces

- Map names to objects, called attributes
  - Implemented as dicts
  - Names can be built-in, global, local to a function, etc.
  - Access via `namespace.attribute`
  - All the constructors for our data structures are part of the `__builtin__` namespace!

# Exceptions

- Exceptions are errors detected during program execution
  - Different from syntax errors, which are detected by the Python interpreter before runtime
- We've seen examples of built-in exceptions: `NotImplementedError` from assignments

# Handling Exceptions

- Exceptions are handled using try except blocks
- Like Java's try-catch blocks
- "It's better to ask forgiveness than permission"

# Handling Exceptions

```
try:  
    # code here that may run into an exception  
except:  
    # this runs if an exception is raised in the "try" block  
    # skipped if no exception is raised during then
```

- Try to put as few lines in “try” - limit the possible number of exceptions that may come up
- If an exception occurs, the rest of the “try” block is skipped
- If exception type match is found, that block is executed and execution continues
- If no match for the Exception is found, execution is stopped with error message

# Handling Exceptions

```
def divide(x, y):  
    try:  
        result = x / y  
  
    except ZeroDivisionError:  
        # won't execute for any other type of exception  
        print("division by zero!")  
  
    else:  
        # only enters here if no exceptions raised  
        print("result is", result)  
  
    finally:  
        # executes regardless of outcome, allows for good code cleanup  
        print("executing finally clause")
```

# Raise Exceptions

- The raise keyword is used to force a specific exception to occur

```
def your_name(name):  
    if not isinstance(name, str):  
        raise Exception('{} is not a string'.format(name))  
  
    print('Hi {}!'.format(name))
```

# Creating Our Own Exceptions

- Easily done by subclassing the base Exception class

# List Comprehensions

- Code is posted on the website calendar
- Click on the highlighted text (“Format, Exceptions, and Comprehensions”) then click “**Open with Google Colaboratory**” on the top right

# Conclusion

- The introduction of keyword arguments allows for optional arguments
- The tuple/dict (un)packing operators allow for any number of args to be passed in
- Keep an eye out for namespaces for next week's lecture
- Exception handling and raising a lot like Java/C++
- Comprehensions for compact sequence generation

**CREDITS:** This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)